

Laboratory Exercise 07

Topics

1. Lists
2. Tables

Discussion

- A. Discuss similarities and differences between lists and tables
- B. Describe the use of the indexes in:
 - a. strings;
 - b. lists;
 - c. tables.
- C. How can you prevent an `out-of-range error`?

Esercizi

Part 1 – List and tables elaboration

Task: For each of the following exercises, write a program in Python that meets the given requirements. Complete at least two exercises during the laboratory session, and the remaining ones at home.

7.1.1 Lists union. Write a program that merges two lists `a` and `b`, alternating one element of the first and one element of the second. If one list is shorter than the other, the items are alternated as long as possible, then the items left in the longer list are added, in order, to the bottom. Starting lists should not be changed. If, for example, the content of `a` is `1 4 9 16` and the content of `b` is `9 7 4 9 11`, the program prints a new list composed of the following values: `1 9 4 7 9 4 16 9 11`. [P6.30]

7.1.2 List of integer numbers. Write a program that reads from input a list of positive integer numbers provided on single row and separated by the character `:`, for example : `3:12:21:8:4:7`. The program must print, maintaining the same format:

- I. The input numbers, excluding the maximum and the minimum (for example, `12:8:4:7`);
- II. Among the input numbers, only the even values (for example, `12:8:4`);
- III. Among the input numbers, only the 2-digits numbers (for example, `12:21`).

7.1.3 Table. Write the instructions for the execution of the following operation for a table in Python of m rows and n columns (dimensions are inserted by the user):

- I. Initialize the table with values equal to zero (`0`);
 - II. Fill all the cells with values equal to one (`1`);
 - III. Fill the cells alternating `0` and `1` in a chess scheme;
 - IV. Fill with `0` only the cells of the upper row and of the lower one, leaving the rest of the table unchanged;
 - V. Fill with `1` only the cells of the right column and of the left column, leaving the rest of the table unchanged;
 - VI. Calculate and display the sum of all the elements.
- After each operation, display the table. [R6.28]

7.1.4 Buffer. Write the pseudocode for an algorithm that, given a list of defined length, moves the elements "forward" one position (thus increasing their index by one unit), and moves the element at the last position to the first position. For example, the list `1 7 9 3 0 4`, after this operation, becomes the list: `4 1 7 9 3 0`. Write a program implementing the above described algorithm. [R6.17]

7.1.5 Sorted lists union. Write a program that merges two lists (which are assumed to be already sorted ascending), returning a new, ascending sorted list. Manage a current index for each list to keep track of portions already processed. Starting lists should not be modified. If, for example, the content of `a` is `1 4 9 16` and the content of `b` is `4 7 9 9 11`, the program prints a new list composed of the following values: `1 4 4 7 9 9 9 11 16`. The methods `sort()` and `sorted()` must not be used. [P6.31]

7.1.6 Out-of-range. Write a program that contains an `out-of-range error`. What happens if you run the program? [R6.10]

Parte 2 – Algorithms using lists and tables

Task: For each of the following exercises, write a program in Python that meets the given requirements. Complete at least one exercise during the laboratory session, and the remaining ones at home.

7.2.1 On average. Write a program that, in a table `values`, calculates the average value of an element's neighbors in the eight directions, indexed as shown in the figure below (excluding the element itself). If, however, the element is on an edge of the table, the average should be calculated by considering only those neighbors that actually belong to the table. For example, if `row` and `column` are both `0`, the element has `3` neighbors. [P6.23]

<code>[i - 1] [j - 1]</code>	<code>[i - 1] [j]</code>	<code>[i - 1] [j + 1]</code>
<code>[i] [j - 1]</code>	<code>[i] [j]</code>	<code>[i] [j + 1]</code>
<code>[i + 1] [j - 1]</code>	<code>[i + 1] [j]</code>	<code>[i + 1] [j + 1]</code>

7.2.2 The theater. A theater seating chart is implemented as a table of ticket prices, like this:

```
10, 10, 10, 10, 10, 10, 10, 10, 10, 10
10, 10, 10, 10, 10, 10, 10, 10, 10, 10
10, 10, 10, 10, 10, 10, 10, 10, 10, 10
10, 10, 20, 20, 20, 20, 20, 20, 10, 10
10, 10, 20, 20, 20, 20, 20, 20, 10, 10
10, 10, 20, 20, 20, 20, 20, 20, 10, 10
20, 20, 30, 30, 40, 40, 30, 30, 20, 20
20, 30, 30, 40, 50, 50, 40, 30, 30, 20
```

30, 40, 50, 50, 50, 50, 50, 50, 40, 30

Write a program that prompts users to pick either a seat or a price. Mark sold seats by changing the price to 0. When a user specifies a seat, make sure it is available. When a user specifies a price, find any seat with that price. [P6.27]

7.2.3 Magic squares. A $n \times n$ matrix containing the integer numbers 1, 2, 3, ..., n^2 is a “magic square” if the sum of its elements in each row, in each column, and in the two diagonals is the same. For example, this is a magic square of size 4:

16	3	2	13
5	10	11	8
9	6	7	12
4	15	14	1

Write a program that acquires by the user 16 values, and places them in a table of 4×4 in order, one row at a time from top to bottom, and in each row from left to right, and check whether, after placing them, they form a magic square. Verify two properties:

- I. All and only the numbers 1, 2, ..., 16 are present in the acquired data.
- II. When the numbers are placed in the table, the sums of the rows, columns and diagonals are all equal to each other. [P6.21]

7.2.4 Tic-tac-toe. Write a program that plays the tic-tac-toe game. The game of tic-tac-toe is played on a 3×3 grid. The game is played by two human players taking turns. The first player marks the moves with a circle ('o'), the second one with a cross ('x'). The player who has formed a horizontal, vertical or diagonal sequence of 3 equal symbols. The program must, at each turn, display the game board, ask the user for the coordinates of the next move symbol (row and column, numbered from 1 to 3) as input, reverse the players after each move, and, when the game is over, decree the winner or a tie condition. [P6.28]

x		o
	x	o
o	o	x

7.2.5 Play dice. Write a program that generates a sequence of 20 random rolls of a dice, stores them into a list and displays the generated values marking the longest set of identical values with the following format:

1 2 5 5 3 1 2 4 3 (2 2 2 2) 3 6 5 5 6 3 1

If there are multiple sequences of identical values of maximum length, highlight the first in order of appearance. [P6.16]

7.2.6 List of functions. Write list functions that carry out the following tasks for a list of integers. For each function, provide a test program. All the operations must modify the list passed as a parameter. [P6.4]

- I. Swap the first and last elements in the list.
- II. Shift all elements by one to the right and move the last element into the first position. For example, `1 4 9 16 25` would be transformed in `25 1 4 9 16`.
- III. Replace all even elements with `0`.
- IV. Replace each element except the first and last by the larger of its two neighbors.
- V. Remove the middle element if the list length is odd, or the middle two elements if the length is even.
- VI. Remove the middle element if the list length is odd, or the middle two elements if the length is even.
- VII. Return the second-largest element in the list.
- VIII. Return `True` if the list is currently sorted in increasing order.
- IX. Return `True` if the list contains two adjacent duplicate elements.
- X. Return `True` if the list contains duplicate elements (which need not be adjacent).

7.2.7 Hidden rules. Write a program that, given an empty list `l = []`, initializes it with each of the following sequences, after you have understood the rules used to generate them: [R6.1]

- I. `1 2 3 4 5 6 7 8 9 10`
- II. `0 2 4 6 8 10 12 14 16 18 20`
- III. `1 4 9 16 25 36 49 64 81 100`
- IV. `0 0 0 0 0 0 0 0 0 0`
- V. `0 1 0 1 0 1 0 1 0 1`
- VI. `0 1 2 3 4 0 1 2 3 4`

7.2.8 Bar chart. Write a program that reads a sequence of float values as input, together with a label, and displays a bar chart of the values, using asterisks (*), like this:

```
Egypt *****
France *****
Japan *****
Uruguay *****
Switzerland *****
```

First figure out the maximum value. That values' bar should be drawn with maximum `40` asterisks. Shorter bars should use proportionally fewer asterisks. The program should work correctly also with negative numbers. [P6.24]